Department of Computer Science and Engineering
Bangladesh University of Engineering and Technology

Level 1, Term II B. Sc. Engineering Examination, Term: January, 2025
Date: July 26, 2025
QUIZ on CSE 106 (Data Structures and Algorithms I Sessional)

Time: 40 Minutes **Full Marks: 50**

Student ID:

Signature of the Invigilator **Marks Obtained:**

Instructions:

1. There are **10 (TEN) QUESTIONS** and **08 (EIGHT) PAGES** in this question booklet. Please count them **before writing your Roll NUMBNER** on the top right corner of this page.
2. All questions carry equal marks.

(Your questions begin below this line.)

---

1. Correct the code provided below and specify the output of the corrected version. Kindly adhere to the following guidelines:

   A. Please underline the portions of the code where corrections have been made.

   B. You must not remove any line of code. However, you may replace lines with corrected code.

```
#include <stdio.h>
int main() {
    int arr[5];
    int i;
    for(i = 0; i <= 5; i++) {
        arr[i] = i * 10;
    }

    int newArr[] = {1, 2, 3, 4, 5};
    arr=newArr //copy elements

    int *a[5]=arr;
    for(i = 0; i < 5; i++) {
        printf("%d ", (a+i));
    }
    return 0;
}
```

2. Write the pseudocode of an algorithm that, given a set S of n integers and another integer x, determines whether S contains two elements that sum to exactly x. Analyze the runtime of your algorithm.

3. Write a function that will take a pointer to the head of a singly linked list and print the list in reverse order in no more than $O(n)$ time. For example, if the list is <10 100 39 21 8>, your function should print "8 21 39 100 10" (without the double quotes). Your function cannot modify the list or use any extra list/array.

4. A postfix expression is one in which every operator follows all of its operands, eliminating the need for parentheses to define evaluation order. Consider "2 3 +" instead of (2 + 3). Here the operator "+" follows the operands "2" and "3". The operator always operates on the two most recent values. Now **write a function postfixEvaluate(expr)** that takes a string **expr** representing a valid postfix expression **containing single-digit non-negative operands** and the **operators addition, subtraction and multiplication (+, -, *)**. The function will return the integer result.

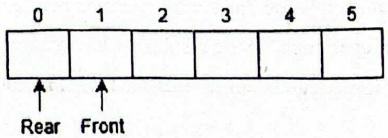
For example: **Input:** "2 3 1 * + 9 -" **Output:** -4

Explanation:

2 3 1 * + 9 -
2 (3*1) + 9 - (Operator *, Operand 3, 1)
2 3 + 9 -
(2+3) 9 - (Operator +, Operand 2, 3)
5 9 -
(5-9) (Operator -, Operand 5, 9)
-4

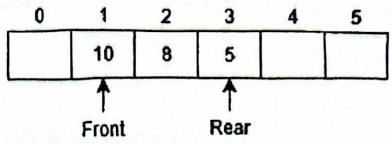
(**Hint**: Use a stack)

5. **Initial Setup:** Below is the underlying array of the ArrayQueue data structure with an initial capacity of 5. Assume **circular array** implementation. The Front and Rear indices are initialized as shown.



First Set of Operations: Apply the following operations to the initial queue and observe the resulting array:

- enqueue(10)
- enqueue(8)
- enqueue(5)



Second Set of Operations: Apply the following operations to the current queue and draw the resulting array.

- enqueue(9)
- dequeue()
- enqueue(7)
- enqueue(11)

TODO: Draw the resulting array below, indicating the positions of the Front and Rear indices

Third Set of Operations: Apply the following operations to the queue obtained after the second set and draw the resulting array.

- enqueue(12)
- enqueue(14)

TODO: Draw the resulting array below, indicating the positions of the Front and Rear indices

Notes:

- The operations are incremental. Apply the 2nd set of operations on the resulting array obtained after the 1st set. Similarly, apply the 3rd set of operations on the resulting array obtained after the 2nd set.
- During enqueue operation, **if the queue becomes full**, first **double the capacity** and then **perform the enqueue**.

6. Given the **pre-order** traversal sequence of a valid binary search tree [7, 4, 2, 1, 3, 6, 5, 10, 8, 9], **draw the BST.**

7. For an undirected, unweighted graph G, we consider the following edges:

   $\{1, 2\}, \{2, 3\}, \{1, 4\}, \{3, 4\}, \{3, 5\}, \{5, 6\}, \{3, 7\}, \{6, 7\}, \{1, 6\}$

   a. Draw the graph
   b. Represent the graph using an Adjacency list
   c. Count the number of different shortest paths between nodes 6 and 1.

8. Using the basic functionalities of a **max heap** data structure, complete the CPP code snippet below to print the smallest 100 elements on an array of size *n*, where *n* is an arbitrarily large number. Your code must run in **O (*n*)** time.

```
void kSmallestElements(vector<int>& arr) {

//Your code here



}
```

9. You have n items, each with weight wt[i], price **price**[i], and an array **fragile**[] where **fragile**[i] is a positive integer if item i is fragile, or -1 if it is normal. You want to select items to put in a bag of capacity **W** such that the total weight does not exceed **W** and the total price is maximized. A fragile item can only be included if the total number of items in the bag (including itself) is at most its fragility threshold; normal items have no such restriction.

(a) Formulate a dynamic programming recurrence relation to solve this problem, clearly defining all variables used.

(b) State the time and space complexity of your solution.

10. You are tasked with manufacturing two types of tools using two components, A and B, where (1) **Tool 1** requires 1 unit of component A and 2 units of component B and (2) **Tool 2** requires 2 units of component A and 1 unit of component B. Each tool, regardless of type, can be sold for 1$. Given two inputs, a and b **($1 \leq a$, $b \leq 10^9$)** representing the number of available units of components A and B, respectively, derive an optimal approach to calculate the maximum total profit you can achieve by producing and selling the tools based on the availability of the components A and B. For example,

$$a = 10 \text{ and } b = 10, \text{ output} = 6$$

$$a = 8 \text{ and } b = 9, \text{ output} = 5$$

[Writing only the approach or pseudo-code will suffice.]